

Room Designer MVP — Iteration 1

A web-based interior design tool built around a deterministic, user-authored room builder: pick a room shape, set dimensions, add doors & windows, style the surfaces, then furnish with parametric archetypes. Camera/LiDAR capture is an optional suggestion layer — not the spine of the product.

USER-AUTHORED STRUCTURE
FORK, DON'T BUILD 0→1
CAPTURE = SUGGESTOR

Version:

2.0 ·

Date:

June 29, 2026

Supersedes:

v1.0 (capture-first architecture)

Architecture thesis:

The room is authored by the user through a wizard, not reconstructed from a scan. Everything in the pipeline maps to an existing open-source repo.

• Contents

1	What Changed in v2 & Why	03
2	Product Thesis & Architecture	03
3	The Core User Flow (4-Step Wizard + Furnish)	04
4	Build-vs-Fork: Open-Source Repo Map	06
5	Recommended Fork Strategy	08
6	Furniture Archetype Library & Asset Sources	09
7	Collision & Snapping (the part you build)	10
8	The Capture Suggestion Layer (optional)	11
9	Functional Requirements	12
10	Tech Stack & Licensing Watch-outs	13
11	Risks, Traps & Milestones	14

1. What Changed in v2 & Why

v1 made camera capture the spine and tried to reconstruct room structure via plane-fitting on a point cloud. v2 inverts this. The user **authors** the room through a deterministic wizard (shape → dimensions → openings → style), exactly like the reference 4-step flow. The scan becomes an **optional suggestion layer** that pre-fills the wizard. If the scan is absent or wrong, the product still works perfectly — the user just picks manually.

Why this is the right inversion:

- **Deletes the hardest, flakiest subsystem.** Plane-fitting on a phone-scan point cloud breaks on bad lighting, glossy floors, clutter, and thin walls — and when it's wrong, the user gets a crooked room they can't easily fix. A user-drawn plan is never "wrong"; it's exactly as accurate as the dimensions entered.
- **Geometry is clean by construction.** Authored walls are axis-aligned parametric rectangles, which makes recoloring, snapping, and collision trivial.
- **It matches a proven UX.** The reference 4-step wizard (shape presets → dimension drag → doors/windows → wall/floor style) is a known, low-friction pattern.
- **It maps onto existing open-source code.** Every step below corresponds to a repo you can fork rather than build from scratch.

2. Product Thesis & Architecture

Three layers, with the **scene graph** as the contract between them — unchanged from v1, but now populated by the wizard instead of a reconstruction service.

Layer	Role
Room builder (wizard)	Deterministic authoring of structure: shape preset, dimensions, doors/windows, wall & floor materials. Writes parametric walls/floor/ceiling + openings into the scene graph.
Furnish & edit	Place / move / rotate / resize / recolor furniture archetypes on the floor plane, with wall & furniture collision and snapping. Reads & writes the scene graph.
Capture suggestor (optional)	A scan or video that pre-fills wizard choices (shape, rough dimensions, detected furniture). Pure convenience; fully skippable.

CORE DECISION (CARRIED FROM V1, REINFORCED)

Never reconstruct furniture, and now: never reconstruct structure either. Structure is user-authored; furniture is chosen from parametric archetypes. The scene graph remains a decomposed set of addressable nodes (walls, floor, openings, furniture) so every edit is a cheap data operation, not a re-render.

3. The Core User Flow

Five stages: the four-step structure wizard, then furnishing. Each maps to a repo in §4.

STEP 1 OF 4

Set the shape and size

User selects a room footprint from presets: **Rectangular, L-Shape, Cut, T-Shape, U-Shape, Beveled**. The 3D preview updates live. Each preset is a predefined polygon of corners; this avoids any free-form drawing for the common case while still covering non-rectangular rooms.

Maps to: room-preset polygons (build a thin preset table) feeding a floor-plan model — blueprint3d `model` / `floorplanner`, or open3dFloorplan "room presets".

STEP 2 OF 4

Adjust your dimensions

User drags wall segments or types exact lengths; live dimension labels with a **feet / centimetres toggle**. Walls stay axis-aligned; corners are draggable handles. This is the parametric heart of the builder.

Maps to: 2D wall/corner editing — blueprint3d `floorplanner`, `react-planner`, or `threejs-room-planner` (all support move/draw/delete walls & corners by mouse). Unit toggle exists in `OpenPlan3D` / `open3dFloorplan`.

STEP 3 OF 4

Add doors and windows

User places openings from a style palette: door styles (single panel, glass, French double, double panel, bifold double, glass double) and window styles (single, double). Each opening snaps onto a wall and can be moved or deleted.

Maps to: door/window placement on walls — `open3dFloorplan` ships single/double/sliding/pocket/bifold/French doors + casement/bay/picture windows out of the box; `blueprint3d` items support wall-mounted openings.

STEP 4 OF 4

Choose your room style

User picks **wall color** (swatch palette) and **floor style** (wood/tile/concrete/etc. textures). Because surfaces are parametric, this is a pure material swap.

Maps to: material editor — `open3dFloorplan` applies textures to walls/floors/ceilings (wood, tile, marble, carpet, concrete, brick); `blueprint3d` `three` view supports per-surface materials.

FINAL STAGE

Furnish the room

User adds furniture from the archetype library, drags it onto the floor plane, rotates, resizes (clamped), and recolors. Items collide with walls and (softly) with each other, and snap to walls. This is where the differentiator lives: the catalog is built from **the user's own furniture archetypes**, not a retailer's.

Maps to: item placement/transform — blueprint3d **items** + **three**, react-planner catalog, threejs-3d-room-designer product list. Collision/snapping = the bespoke piece (see §7).

UX NOTE

Keep the wizard linear with Back/Next, a persistent live 3D preview on the right, and the option to jump straight into Furnish from a saved room. Offer "start from a scan" as an entry point that pre-fills Steps 1–2 — never as a requirement.

4. Build-vs-Fork: Open-Source Repo Map

The point of this section: **you are not building 0→1**. Below is every viable repo found, what it gives you, its stack, and its license. Verify licenses against the live repos before committing — they can change.

Repo	What it gives you	Stack	License*	Best used as
furnishup/ blueprint3d	The canonical browser room planner: 2D floorplan editor, shared data model, items module, 3D view. Architecture = your scene graph.	JS / Three.js (some jQuery in example)	Open-source (see LICENSE.txt)	Primary architecture reference / core fork
blueprint3d ES6/ React fork	Modernized ES6 rewrite of blueprint3d; React wrappers exist (e.g. blueprint-3d configurator wrapped with React).	ES6 / React / Three.js	Inherits blueprint3d	Closest to your React stack
theLodgeBots/ open3dFloorplan	Most feature-complete: room presets, snap walls, full door/window styles, material editor, 2D→3D, GLB import, undo/redo, unit toggle.	SvelteKit / Three.js	MIT	Best UX/feature reference; port ideas
OpenPlan3D	Walls/rooms/doors/windows with measurements, snap-to-grid, 2D→3D walkthrough, furniture library, GLB import, multi-unit, and <i>LIDAR room import</i> (your "scan as helper").	Web / Three.js	Open source (confirm on repo)	Reference for the capture-suggestor flow
cvdlab/react-planner	React component: draw 2D floorplan, navigate in 3D, drag-drop from an extensible catalog you populate with your own objects.	React / Three.js	Permissive (confirm)	React-native fork w/ catalog system
threejs-room-planner / threejs-3d-room-designer	React + Three.js: move/draw/delete walls & corners in 2D, add models, change position/orientation in 2D and 3D.	React / Three.js	Confirm on repo	Lightweight on-stack starting point
mehanix/arcada	React + Pixi.js floor-plan creator, custom engine, Zustand state, Express/Mongo backend.	React / Pixi.js	Confirm on repo	2D drawing-UX reference

*Licenses must be re-verified on the actual repository before use — especially for anything deployed commercially. open3dFloorplan and its npm package are MIT; blueprint3d ships a LICENSE.txt; others require checking.

5. Recommended Fork Strategy

RECOMMENDATION

Fork a **React + Three.js** base whose architecture already separates 2D-floorplan / data-model / items / 3D-view — i.e. the blueprint3d lineage (ideally the ES6/React fork) or react-planner. Then lift the richest features (room presets, full door/window styles, material editor, unit toggle, GLB import, LiDAR import) from **open3dFloorplan** and **OpenPlan3D**, which are MIT/open and already implement exactly the four wizard steps.

Why not just adopt open3dFloorplan wholesale?

It is the most complete and MIT-licensed, but it is **SvelteKit**. If the product is React (assumed, given the configurator direction), porting the whole app is more friction than forking a React base and porting its *features and logic*. If you are framework-agnostic, adopting open3dFloorplan directly is the fastest path to feature parity.

What you will still write yourself

- **The shape-preset table** (Step 1): a small set of corner polygons for Rectangular/L/T/U/Cut/Beveled. Trivial data, not in any one repo as a clean "preset picker."
- **Collision & snapping** (§7): none of the planners ship robust furniture-vs-wall collision; this is the known gap.
- **The parametric archetype + clamped-resize behavior** (§6): most catalogs place static models; resizable archetypes with sane bounds are your addition.
- **The capture suggestor glue** (§8): wiring an optional scan to pre-fill the wizard.

6. Furniture Archetype Library & Asset Sources

The library is the moat — the user decorates with archetypes that match *their* furniture, then resizes and recolors. Cover the head of the distribution (~10–15 categories) plus a placeholder box for the long tail.

6.1 Category coverage (v1)

Category	Representative archetypes
Sofa	3-seater, loveseat, L-shaped sectional, recliner, chaise
Bed	single, double/queen, king (with/without headboard)
Table	coffee, dining (rect), dining (round), side
Chair	dining, armchair, office, accent
Storage	wardrobe, shelf/bookcase, TV unit, dresser
Decor	rug, floor/table lamp, potted plant
Fallback	generic placeholder box, user-sized, for the long tail

6.2 Asset sources (bootstrap vs own)

Source	What it offers	Use / caveat
FurniMesh	Thousands of free web-ready GLB furniture models (sofa/chair/table/bed), single self-contained .glb.	Fast bootstrap; verify per-model license terms.
Poly Pizza / KayKit / itch.io low-poly packs	Low-poly furniture packs in GLB/FBX, often CC0 or permissive.	Great for a consistent stylized v1 look.
CC0 packs (CGTrader CC0, Blender3DArchitect CC0, ITHappy free)	PBR-textured CC0 furniture with real-world scale; some include collision.	Public-domain-safe; check each listing.
Commission / in-house Blender	Full control of parametric topology & single editable PBR material per surface.	Best for clean resize + color swap; slower/cost.

ASSET REQUIREMENTS

Require glTF/GLB export, clean topology compatible with the chosen scaling strategy, real-world scale, and one editable PBR material slot per recolorable surface. Bootstrap with free GLB packs to validate the flow, then commission a consistent set — mixed-topology bargain packs cost more in integration than they save.

7. Collision & Snapping — the part you build

This is the one subsystem no planner repo ships robustly (it's a recurring question in the Three.js community: TransformControls happily lets objects pass through walls). You build it — but it does **not** require a physics engine, which would be overkill and add instability.

Approach: 2D footprint constraint in the floor plane

Because authored walls are axis-aligned and furniture sits flat on the floor, treat collision as a 2D problem and ignore height. Each item has a footprint rectangle; each wall is a segment/rectangle. On every drag tick:

1. Compute the proposed footprint at the cursor target.
2. Test it against wall segments and against other items' footprints (AABB; or oriented-box / separating-axis test once rotation is allowed).
3. If it would cross a wall, **clamp** to the last valid position and let the item *slide* along the wall rather than stopping dead — this feels far better.

Refinements to build in from the start

- **Wall snapping:** when an item's back edge is within a few cm of a wall, snap it flush. This is what makes a sofa sit against the wall instead of floating.
- **Soft furniture overlap:** make item-vs-item overlap a warning (highlight red) rather than a hard block — people legitimately tuck a side table under a desk.
- **Rotation-aware footprints:** recompute the rectangle on rotate; switch AABB→OBB/SAT for non-axis-aligned angles.
- **Opening awareness (nice-to-have):** optionally warn when furniture blocks a door swing.

8. The Capture Suggestion Layer (optional)

Capture is now strictly a **helper**. Its job is to pre-fill the wizard so the user confirms instead of authoring from scratch. The product must be fully usable with capture turned off.

What capture can suggest

- **Shape & rough dimensions** (pre-select Step 1 preset + pre-fill Step 2). LiDAR scan (as in OpenPlan3D's import) is the cleanest source where available; a guided video is a fallback.
- **Detected furniture** (pre-fill the furnish picker with category + dominant color). A lightweight 2D detector trained on the HomeObjects-3K dataset (bed, sofa, chair, table, lamp, tv, laptop, wardrobe, window, door, potted plant, photo frame) covers the common categories and trains with a single command. Convenience only.

Design rules for the suggestor

- Every suggestion is editable; nothing is locked.
- Wrong suggestions degrade to a one-tap correction, never a broken room.
- No heavy reconstruction on the critical path — if a scan needs cloud processing, it runs async and the user can proceed manually meanwhile.

HONESTY OF PROMISE

The room is an accurate authored model in the user's chosen style; placed furniture is the matching archetype in their color and size — not a literal twin of their specific pieces. Don't market it as a digital twin.

9. Functional Requirements

ID	Requirement	Priority	Source
FR-1	User selects a room shape from presets (Rect, L, T, U, Cut, Beveled) with live 3D preview.	Must	Build (preset table)
FR-2	User adjusts wall lengths by drag or typed value; live labels; ft/cm toggle.	Must	Fork (floorplanner)
FR-3	User places/moves/deletes doors & windows from a style palette, snapped to walls.	Must	Fork (open3dFloorplan)
FR-4	User picks wall color and floor texture (material swap).	Must	Fork (material editor)
FR-5	User adds furniture archetypes onto the floor plane.	Must	Fork (items/catalog)
FR-6	User moves, rotates, and resizes furniture (clamped scaling).	Must	Fork + build (resize)
FR-7	Furniture cannot pass through walls; slides/clamps on collision.	Must	Build (\$7)
FR-8	Furniture snaps flush to walls within a threshold.	Should	Build (\$7)
FR-9	Furniture-vs-furniture overlap warns (soft), does not hard-block.	Should	Build (\$7)
FR-10	User recolors / swaps material on any furniture item.	Must	Fork + assets
FR-11	User saves a design and reopens it (scene-graph persistence).	Must	Fork + build
FR-12	Optional: import a LiDAR/video scan to pre-fill shape & dimensions.	Could	Fork (OpenPlan3D)
FR-13	Optional: detector pre-fills furniture picker (category + color).	Could	Build (HomeObjects-3K)
FR-14	Generic placeholder box for unmodeled furniture types.	Should	Build

10. Tech Stack & Licensing Watch-outs

Layer	Recommendation	Rationale
Framework + 3D	React + React Three Fiber (Three.js)	Industry standard for declarative 3D configurators; huge ecosystem (TransformControls, GLTF loaders, Environment, LOD).
Core fork	blueprint3d ES6/React fork or react-planner	Architecture already = scene graph (floorplanner / model / items / three).
Feature donor	open3dFloorplan (MIT), OpenPlan3D	Room presets, door/window styles, material editor, unit toggle, GLB + LiDAR import.
Asset format	glTF 2.0 (.glb)	Standard web 3D; every engine + viewer supports it.
Furniture assets	FurniMesh / CC0 packs → commission later	Bootstrap free, then a consistent commissioned set.
Detector (optional)	Modern YOLO + HomeObjects-3K	Dataset matches target categories; trains in one command. Convenience layer only.

LICENSING WATCH-OUTS (VERIFY BEFORE SHIPPING)

Re-check each forked repo's license on the live repository — open3dFloorplan is MIT; blueprint3d ships LICENSE.txt; others need confirming. Modern YOLO tooling is frequently AGPL-3.0, which carries network-use obligations for a hosted product — confirm a commercially compatible detector. Validate per-model license terms for any free GLB assets; CC0 is safest.

11. Risks, Traps & Milestones

TRAP 1 — PORTING A SVELTE APP INTO REACT BY HAND

open3dFloorplan is the richest, but it's SvelteKit. Don't transliterate it line-by-line into React; fork a React base and port its *logic and UX patterns*. Decide the framework up front.

TRAP 2 — ASSUMING A FORK SHIPS COLLISION

None do, robustly. Budget for \$7 as bespoke work from day one; it's the single most user-visible quality signal in a room planner.

TRAP 3 — FURNITURE SCALING LOOKS WRONG

Naive non-uniform scaling distorts cushions/arms. Ship clamped scaling in v1; plan segmented (cushion-repeat) scaling for v2. Validate visually with real archetypes.

TRAP 4 — LICENSE SURPRISE AT LAUNCH

A permissive-looking repo or asset pack with a restrictive sub-dependency (or AGPL detector) can block commercial deployment. Audit licenses before building on them, not after.

Suggested milestones

Milestone	Deliverable	Definition of done
M0	Fork chosen, scene-graph schema, shape-preset table	React base running locally; presets render in 3D.
M1	Steps 1–2: shape + draggable dimensions + unit toggle	User authors an L/T/U room to exact size.
M2	Steps 3–4: doors/windows + wall/floor styling	Openings placed; surfaces recolored/textured.
M3	Furnish: archetype placement + clamped resize + recolor	User drops, sizes, and recolors a sofa.
M4	Collision + snapping	Furniture clamps at walls, slides, snaps flush.
M5	Save / reopen designs	Scene graph persists and reloads.
M6	Optional capture suggestor	A scan pre-fills shape/dimensions and furniture picker.

One-line build summary. Fork a React + Three.js planner (blueprint3d lineage / react-planner), port room-preset + door/window + material features from open3dFloorplan / OpenPlan3D, add the bespoke collision-and-snapping layer and clamped-resize archetypes, and treat any scan as an optional wizard pre-fill. Almost nothing is built from scratch.

End of document · Room Designer MVP, Iteration 1 · v2.0 · Verify all repo licenses on their live repositories before commercial use.